Problem Statement: Word Search Solver API

Overview

Design and implement a **Word Search Solver API** that detects predefined hidden words from a fixed word-search puzzle and returns their exact positions in a strictly defined response format.

The API will be evaluated using **contract-first automated tests**. Your solution **must match the expected output exactly**, including word order, coordinate values, and JSON structure.

API Endpoint

- **Method:** `POST`
- **Path:** `/api/v1/puzzles/word-search/solve`
- **Input:**
  - No request body parameters are required.
  - The word-search puzzle image is **pre-distributed** and must be **bundled internally** with the solution.
  - The API must not expect image uploads or file paths.

Puzzle Requirements

The puzzle image itself contains a **few words to be searched**, all enclosed in the grid of characters.

- The solver must:
  - Detect **only** these words
  - Ignore any additional or partial matches
  - Treat every request as **self-contained**

Expected Words (example only, not true in real puzzle)

The API **must return exactly the following four words**, sorted **alphabetically** and **in uppercase**:

1. `CAPTAINAMERICA`
2. `HULK`
3. `IRONMAN`
4. `THOR`

No additional words are permitted.

Coordinate System

- Each word must include **per-letter coordinates**
- Coordinates must:
  - Use `{ "x": number, "y": number }`
  - Start from **origin (0,0) at the bottom-left**
  - Be listed **in the sorted order of all the words found** in the puzzle

Response Contract

The response **must** conform to the following structure:

```
{
 "wordsFound": [ "<WORD_1>", "<WORD_2>", "<WORD_3>", "<WORD_4>" ],
 "words": {
  "<WORD>": {
   "coordinates": [
    { "x": <number>, "y": <number> }
   ]
  }
 }
}
```

Exact Coordinate Expectations

The solver must return the **exact coordinates** below.

CAPTAINAMERICA

- Horizontal, left to right

```
[
 { "x": 2,  "y": 3 },
 { "x": 3,  "y": 3 },
 { "x": 4,  "y": 3 },
 { "x": 5,  "y": 3 },
 { "x": 6,  "y": 3 },
 { "x": 7,  "y": 3 },
 { "x": 8,  "y": 3 },
 { "x": 9,  "y": 3 },
 { "x": 10, "y": 3 },
 { "x": 11, "y": 3 },
 { "x": 12, "y": 3 },
 { "x": 13, "y": 3 },
 { "x": 14, "y": 3 },
 { "x": 15, "y": 3 }
]
```

Free trial

HULK

- Vertical, top to bottom

```
[
 { "x": 6, "y": 8 },
 { "x": 6, "y": 7 },
 { "x": 6, "y": 6 },
 { "x": 6, "y": 5 }
]
```

IRONMAN

- Diagonal, top-left to bottom-right

```
[
 { "x": 1, "y": 12 },
 { "x": 2, "y": 11 },
 { "x": 3, "y": 10 },
 { "x": 4, "y": 9  },
 { "x": 5, "y": 8  },
 { "x": 6, "y": 7  },
 { "x": 7, "y": 6  }
]
```

THOR

- Horizontal, left to right

```
[
 { "x": 9,  "y": 14 },
 { "x": 10, "y": 14 },
 { "x": 11, "y": 14 },
 { "x": 12, "y": 14 }
]
```

Validation Rules

- ☑ HTTP Status must be 200
- ☑ `wordsFound` must:
  - Contain **exactly four entries**
  - Be **alphabetically sorted**
  - Use **uppercase only**
- ☑ `words` must:
  - Contain matching keys for each word
  - Include **only** the expected coordinates
- ✕ Any deviation in:
  - Word list
  - Sorting
  - Coordinates
  - JSON structure

  will cause test failure

Evaluation Criteria

Your solution will be validated using **Karate DSL feature tests** with:

- Full JSON equality checks
- Exact coordinate matching.
- No tolerance for extra fields or missing data

This is an **exact-output contract challenge**, not a heuristic or fuzzy solver.

Free trial